

# MyFamilyTree — Diseño v1

**Fecha:** 2026-09-21 · **Estado:** Aprobado para pasar a plan de implementación

---

## 1. Propósito y visión

---

MyFamilyTree es un producto (no solo una herramienta personal) para construir árboles genealógicos de forma colaborativa. Cada usuario empieza su propio árbol y, mediante invitaciones explícitas, los árboles de distintos familiares pueden conectarse hasta formar un grafo familiar compartido — incluyendo personas fallecidas, documentadas por quien las recuerda.

La visión a largo plazo es unificar de forma responsable la jerarquía entre generaciones de una familia extendida, con un mecanismo de fusión de árboles que evite duplicados y respete la privacidad de cada rama. Esta v1 sienta las bases de la colaboración (invitación + permisos de edición); la detección automática de duplicados y el consenso multi-familiar para editar personas fallecidas quedan para v2.

## 2. Alcance de la v1

---

### Incluido

- Registro/login con Google OAuth o magic link por correo.
- Crear y visualizar un árbol genealógico propio.
- Agregar personas (vivas o fallecidas) con relaciones padre/madre-hijo.
- Invitar a un familiar vivo por correo, vinculado a un nodo específico del árbol.
- Al aceptar la invitación, el familiar se une como colaborador de ese árbol compartido.
- Cualquier colaborador puede proponer ediciones a cualquier nodo; el creador del nodo aprueba antes de que el cambio se aplique. Historial de cambios visible.
- Fotos de perfil por persona (Supabase Storage).

### Explícitamente fuera de alcance (v2+)

- Detección automática de posibles duplicados entre árboles no conectados.
- Consenso/votación multi-familiar para editar personas fallecidas con varios descendientes conectados.
- Relación "pareja/matrimonio" independiente (sin hijos en común).
- Búsqueda pública o descubrimiento de árboles/personas.
- Galería multimedia múltiple por persona, eventos de vida estructurados, biografía/notas extensas.
- App nativa iOS publicada (la arquitectura lo soporta, pero no se publica en v1).

### 3. Modelo de datos y relaciones

---

#### Persona ( `people` )

- `id`, `full_name`, `photo_url` (opcional), `birth_date` / `birth_place` (opcionales), `death_date` / `death_place` (opcionales), `is_living` (boolean).
- `attributes` (JSONB): campo abierto para datos futuros (enfermedades, ocupación, etc.) sin requerir migraciones de esquema para cada campo nuevo.
- `created_by`: usuario que creó el nodo — determina quién aprueba ediciones.
- `claimed_by_user_id` (nullable): cuenta de usuario que reclamó este nodo al aceptar una invitación. Nulo mientras el nodo es un placeholder.

#### Relaciones ( `relationships` )

- `parent_id`, `child_id`, `type` ( `biological` | `adoptive` ).
- Es la única primitiva de relación. No existe una arista "pareja/cónyuge" en v1: dos personas se reconocen como pareja implícitamente por compartir un hijo. Relaciones como tío, primo o medio-hermano se calculan recorriendo el grafo (consultas recursivas), no se almacenan directamente — excepto medio-hermanos, que emergen naturalmente de compartir un solo padre en las aristas existentes.
- Se debe prevenir ciclos: ninguna persona puede ser su propio ancestro (validación al crear una relación).

#### Invitaciones ( `invitations` )

- `person_id` (nodo placeholder al que apunta), `email`, `invited_by`, `status` ( `pending` | `accepted` | `expired` ), `tree_id`.
- Al aceptar: si el correo de la cuenta que acepta coincide con el invitado, el nodo pasa a `claimed_by_user_id = <usuario>` y se crea una fila en `tree_memberships`.

#### Membresías ( `tree_memberships` )

- `tree_id`, `user_id`, `joined_at`. Determina quién puede ver y colaborar en un árbol. Una vez que un usuario reclama un nodo y se une a un árbol vía invitación, sigue contribuyendo a ese árbol compartido (no bifurca un árbol paralelo con la misma identidad). Si nunca acepta ninguna invitación, es libre de crear su propio árbol independiente en cualquier momento.

#### Cambios pendientes ( `change_requests` ) y auditoría ( `change_history` )

- `change_requests`: `person_id`, `proposed_by`, `proposed_changes` (JSONB diff), `status` ( `pending` | `approved` | `rejected` ), `created_at`.
- Solo el `created_by` del nodo (o quien herede ese rol, ver §6) puede aprobar/rechazar.
- `change_history`: registro inmutable de cada cambio aplicado — quién propuso, quién aprobó, qué cambió, cuándo. Visible para todos los colaboradores del árbol.

## 4. Flujo de colaboración

---

1. Usuario A se registra (Google o magic link) → se crea automáticamente su propio nodo de persona, reclamado por su cuenta, y un árbol nuevo ( `trees` ) del cual es miembro.
2. A agrega un nodo placeholder para su hermana, con relación padre/madre compartida, y asocia el correo de su hermana.
3. Se envía una invitación a ese correo, apuntando a ese nodo específico.
4. La hermana recibe el link, inicia sesión (Google o magic link) con ese correo.
5. El sistema valida que el correo coincide con la invitación → el nodo queda `claimed_by_user_id = hermana` , y se crea su `tree_membership` en el árbol de A.
6. La hermana ahora puede: agregar más nodos a ese árbol compartido (p. ej. agregar a la abuela materna fallecida), y proponer ediciones a cualquier nodo existente.
7. Toda edición que la hermana proponga sobre un nodo que A creó queda pendiente hasta que A la apruebe (y viceversa).

Sin invitación aceptada, no hay conexión entre árboles — esto es intencional: v1 no intenta detectar automáticamente que dos árboles separados describen a la misma familia.

## 5. Privacidad y acceso

---

- Sin búsqueda pública ni descubrimiento: el único camino de acceso a un árbol es una invitación directa por correo a un nodo específico.
- Un colaborador ve y puede editar (proponer cambios sobre) todo el árbol al que pertenece vía `tree_memberships` — no hay niveles de visibilidad parcial por rama en v1 (esa idea de "mi familia" configurable por rama se descarta a favor del modelo de invitación, más simple y predecible).

## 6. Casos límite conocidos (no bloqueantes para v1)

---

- **Creador inactivo/cuenta eliminada:** sus `change_requests` pendientes quedan sin poder aprobarse. Mitigación diferida a v1.1 (p. ej. reasignar aprobación al colaborador más antiguo del árbol tras un periodo de inactividad). Documentado como limitación conocida, no bloquea el lanzamiento.
- **Duplicados no conectados:** si dos personas describen al mismo familiar sin invitarse mutuamente, existirán dos nodos separados hasta que alguien envíe una invitación que los una. Aceptado como limitación de v1 (la detección automática es v2).
- **Pareja sin hijos en común:** no se puede representar en v1 (ver §2).

## 7. Stack técnico

---

- **Frontend:** Expo (React Native + React Native Web) en TypeScript. Una sola base de código sirve Web y Android en v1; iOS queda soportado por la misma arquitectura y se activa en una fase posterior sin reescritura.

- **Backend:** Supabase — Postgres administrado, Auth (Google OAuth + magic link), Storage para fotos de perfil. Evita operar un servidor propio en v1.
- **Hosting:**
  - Web: Vercel, con auto-deploy desde GitHub en cada push a `main`.
  - Android: Expo EAS Build (tier gratuito) para generar el APK/AAB.
  - Todo en tiers gratuitos para el lanzamiento inicial.
- **Base de datos:** Postgres con consultas recursivas (CTEs) para navegar el grafo familiar (ancestros, descendientes, relaciones calculadas como tío/primo). No se usa una base de datos de grafos dedicada en v1 — la escala esperada no lo justifica.

## 8. Testing

---

- **Unit (Jest):** prevención de ciclos en relaciones, matching de invitaciones por correo, máquina de estados de `change_requests` (pending → approved/rejected).
- **Integración:** pruebas contra Postgres/Supabase real (o local) para la capa de datos y políticas de acceso (row-level security por `tree_membership`).
- **E2E:** flujo feliz completo — registro → crear árbol → agregar placeholder → invitar → aceptar invitación con otra cuenta → colaborar → proponer y aprobar un cambio.

## 9. Preguntas abiertas para v2 (fuera de esta spec)

---

- Diseño concreto del algoritmo de detección de duplicados (nombre + fechas + conexión compartida) y su UI de sugerencia/confirmación.
- Mecanismo de consenso multi-familiar para editar personas fallecidas con varios descendientes conectados (cuántos votos, quórum, desempates).
- Soporte para relación "pareja" sin hijos en común.
- Publicación de la app en iOS.

## 10. Adenda — carga incremental del árbol (2026-09-21)

---

El árbol crecerá con el tiempo a través de generaciones y fusiones de árboles vía invitación, por lo que la aplicación no debe cargar el grafo completo de una sola vez. Esto es un requisito no funcional de v1, no algo diferible a v2:

- **API/consultas:** las consultas recursivas (CTEs) sobre `relationships` deben acotarse por profundidad/ventana (p. ej. N generaciones arriba/abajo desde el nodo enfocado), nunca traer el árbol completo en una sola llamada.
- **UI:** la vista del árbol carga y renderiza solo la porción visible/cercana al nodo actual, y solicita más datos (ancestros, descendientes, ramas colapsadas) a medida que el usuario navega o expande un nodo.

Esta restricción debe reflejarse en el plan de implementación tanto en el diseño de los endpoints/queries de Supabase como en los componentes de visualización del árbol.